

A Zero-Knowledge Lifecycle State Machine for Digital Product Passports

Anonymous Submission

Abstract

Supply-chain traceability requires guaranteeing the correctness of multi-stage transformations while preserving confidentiality of intermediate material states. Existing systems that combine blockchains and zero-knowledge proofs typically treat blockchains as immutable logs and apply zero-knowledge proofs only to per-event attribute validation. As a result, they leave fundamental correctness gaps such as double consumption, broken multi-hop lineage, and the inability to verify split/merge transformations while concealing intermediate values.

We formalize supply-chain traceability as a *state-transition system* in which a blockchain enforces global non-equivocation via nullifiers, and zero-knowledge proofs verify the correctness of each local transformation. Our model supports produce/consume, split/merge, and arbitrary transformations, while enabling selective disclosure of only final aggregates such as CO₂ totals. We implement a prototype in Rust using the **arkworks** ecosystem with [Plonk+KZG over BLS12-381](#), and [evaluate proving time, proof size, and on-chain verification gas cost on an EVM-compatible chain](#).

1 Introduction

Regulatory and industrial initiatives such as the EU Digital Product Passport (DPP) mandate that manufacturers provide detailed, verifiable, and machine-readable lifecycle information for products, including material origin, processing history, environmental impact, and end-of-life handling. Achieving this vision requires two properties that are difficult to satisfy simultaneously: (1) *end-to-end verifiability* of all lifecycle transformations and (2) *privacy preservation* for sensitive industrial data such as formulas, process parameters, or internal supply-chain structures.

Existing DPP implementations rely heavily on centralized or consortium-managed databases, where actors upload lifecycle information in plaintext. Such systems suffer from several limitations:

- **Lack of cryptographic verifiability:** Downstream actors must trust upstream declarations without guarantees of correctness.

- **Poor privacy controls:** Sensitive process details (composition ratios, yields, hazardous material usage) must be revealed to prove compliance, leaking proprietary information.
- **Inconsistent lifecycle histories:** Without a canonical global state, different actors may maintain diverging views of product history.

Blockchain-based approaches promise immutability and transparency, but most existing designs treat the blockchain merely as an *append-only log*: each lifecycle event is recorded as a new log entry. While this guarantees tamper resistance, it does not provide a mechanism to ensure that: (1) inputs are consumed exactly once, (2) lifecycle rules are enforced at every step, or (3) intermediate values remain private. Consequently, many blockchain-based supply-chain systems cannot satisfy the industrial privacy constraints required for real-world DPP adoption.

Key Observation. A correct DPP system must treat the blockchain not as a log, but as a *state machine*. Each lifecycle event is a state transition that consumes existing commitments and produces new ones. The blockchain must enforce one-time consumption, maintain a canonical global state, and verify compliance rules—while zero-knowledge proofs must enforce local correctness without revealing private data.

Our Contribution. We introduce a ZK-enabled lifecycle state machine that integrates commitments, nullifiers, and zero-knowledge proofs into a blockchain-backed global state. Our system allows any actor to perform a lifecycle transformation by consuming input commitments, generating a zero-knowledge proof of correctness, and producing new commitments that represent updated product attributes. The blockchain verifies the proof and updates its state, ensuring that all actors observe a single, consistent lifecycle history.

Our model is ZKP-agnostic and general enough to support diverse lifecycle operations such as material refinement, assembly, repair, disassembly, and recycling. Because sensitive attributes remain committed and all correctness checks occur inside zero-knowledge proofs, the system provides both *strong privacy* and *end-to-end verifiability*—requirements that traditional DPP and blockchain-based systems fail to satisfy.

Technical Contributions. This paper makes the following contributions:

1. We formalize a ZK lifecycle state machine that captures the evolution of products through commitments, nullifiers, and ZKP-verified state transitions.
2. We prove global soundness and acyclicity by combining blockchain consensus with one-time-use semantics enforced by nullifiers.
3. We propose a composability framework enabling complex multi-stage industrial lifecycles to be represented as privacy-preserving and verifiable DAGs.
4. We build a fully functional implementation based on modern succinct zero-knowledge proof systems and evaluate its performance on real-world DPP workloads.

By combining blockchain consistency with ZK-enforced local correctness, our system provides the first general, privacy-preserving, and verifiably correct foundation for Digital Product Passports across diverse industrial domains.

2 Background

This section reviews the technical foundations relevant to our model: Digital Product Passports, commitment schemes, zero-knowledge proofs, and blockchains as global state machines. These components form the basis of the lifecycle transition framework presented in Section 4.

2.1 Digital Product Passports and Lifecycle Traceability

Digital Product Passports (DPPs) are emerging as a regulatory requirement for providing machine-readable, lifecycle-wide information about products. Under frameworks such as the EU Ecodesign for Sustainable Products Regulation (ESPR) and the EU Battery Regulation, manufacturers must report material composition, recycled content, carbon footprint, repair history, and end-of-life processing. Industry initiatives including Catena-X, CIRPASS, and GS1 Digital Link aim to establish interoperable standards for cross-industry sustainability reporting and traceability.

A fundamental challenge in DPP systems is the tension between *verifiability* and *privacy*. Lifecycle chains often span multiple organizations, each holding sensitive data such as process parameters, mixing ratios, or hazardous-material handling information. A correct DPP must therefore allow downstream actors to verify compliance with lifecycle rules without revealing proprietary intermediate values.

2.2 Commitment Schemes

A commitment scheme allows an actor to commit to a hidden value while enabling verifiable computations over that value inside a zero-knowledge proof. A commitment to an attribute vector v with randomness r is denoted:

$$cm = \text{Com}(v, r).$$

The scheme must satisfy:

- **Hiding:** The commitment reveals no information about v .
- **Binding:** The committer cannot later change v without detection.

In lifecycle systems, commitments represent product attributes such as mass, recycled content, and hazard classifications. Commitments appear on-chain, while openings (v, r) remain private and are used only inside ZKP circuits.

2.3 Zero-Knowledge Proofs

Zero-knowledge proofs (ZKPs) allow an actor to prove that a statement holds without revealing the underlying data. Succinct non-interactive zero-knowledge proofs (SNARKs) provide:

- **Succinct verification:** Proofs are small and quickly verifiable.
- **Expressiveness:** Arbitrary arithmetic relations can be encoded in circuits.
- **Zero-knowledge:** No information beyond the statement’s validity is leaked.

A lifecycle transition is encoded as a ZKP that proves:

1. the actor knows openings of the input commitments,
2. the nullifiers are correctly computed,
3. the transformation satisfies domain-specific lifecycle rules,
4. the output commitments are correctly derived.

Our design is ZKP-agnostic: any efficient SNARK or STARK system can be used, and implementation choices (e.g., Groth16) do not affect the state machine semantics.

2.4 Blockchain as a Global State Machine

Blockchains provide a tamper-resistant, consensus-backed global state that is shared across all participants. Traditional supply-chain systems treat blockchains as append-only logs, but lifecycle verification requires stronger semantics.

A blockchain maintains a canonical state $\mathcal{S}$ updated by validated transactions. Consensus guarantees:

- **Consistency:** All honest parties observe the same global state.
- **Finality:** Once confirmed, a state update cannot be reverted.
- **Non-equivocation:** A lifecycle input cannot be consumed differently by different participants.

In our model, the blockchain enforces:

- one-time-use semantics for commitments via nullifiers,
- insertion of new commitments into the global live set,
- verification of ZKPs that ensure local lifecycle correctness.

This combination of blockchain-enforced global consistency and ZKP-enforced local correctness provides the security foundation for verifiable, privacy- preserving Digital Product Passports.

3 Problem Statement and Design Goals

Digital Product Passports require end-to-end traceability across the entire lifecycle of a product, spanning manufacturing, assembly, distribution, use, repair, disassembly, and recycling. Each stage generates information that is both sensitive and essential for regulatory compliance. A correct system must enable downstream actors to verify that upstream lifecycle rules were followed without revealing proprietary process details.

This section formalizes the problem that a secure and privacy-preserving DPP infrastructure must solve.

3.1 Problem Statement

Lifecycle chains in real industrial environments exhibit three critical properties:

1. **Multi-hop transformations:** Products undergo multi-stage transformations in which the output of one actor becomes the input for another. Existing blockchain systems record events but do not enforce state semantics such as single-use consumption or material conservation.
2. **Sensitive intermediate attributes:** Actors possess private data such as material composition, mixing ratios, hazardous material indicators, or yield data. This information must remain confidential while still enabling correctness proofs.
3. **Regulatory consistency:** Regulations require that lifecycle rules (mass balance, CO₂ footprint, recycled content, hazardous-material thresholds) hold across multiple stages. Verification must be end-to-end and cryptographically sound.

Goal. We seek a system that provides global lifecycle correctness, strong privacy guarantees, and selective disclosure while remaining compatible with existing blockchain and industrial workflows.

3.2 Design Goals

A secure and privacy-preserving DPP infrastructure must satisfy the following design goals:

G1. Global State Consistency. All participants must observe a single canonical lifecycle DAG. Inputs must not be consumed more than once, and lifecycle histories must be consistent across all actors.

G2. Local Correctness with Privacy. Actors must prove that each lifecycle transformation satisfies domain-specific rules. Sensitive attributes, intermediate values, and processing specifics must remain hidden.

G3. Composability Across Multiple Stages. Lifecycle proofs must compose across dozens of manufacturing or recycling steps, forming a globally sound end-to-end chain.

G4. Selective Disclosure. Downstream actors should verify only the compliance indicators required for regulation (e.g., recycled-content threshold) without learning internal details from upstream processes.

G5. Auditability and Trust Minimization. The system must eliminate reliance on centralized authorities. Verification should be automated, reproducible, and based solely on on-chain state and cryptographic proofs.

3.3 Threat Model

We assume actors in the lifecycle may behave adversarially. The system must remain correct under the following threat conditions:

- **Dishonest producers or manufacturers:** Attempting to falsify material composition, recycled content, emissions, or hazardous-material indicators.
- **Dishonest intermediaries:** Attempting to reuse inputs across multiple transitions, forge process claims, or hide violations of lifecycle constraints.
- **Collusion:** Multiple upstream actors collaborating to produce an inconsistent or fraudulent lifecycle history.
- **Curious downstream actors:** Attempting to infer proprietary manufacturing data through repeated queries or analysis of released lifecycle information.
- **Blockchain adversaries:** Capable of observing all on-chain data but unable to break standard cryptographic assumptions or the chain’s consensus safety.

We assume the underlying blockchain provides consensus, data availability, finality, and collision-resistant hash functions. Zero-knowledge proof systems provide soundness, completeness, and zero-knowledge properties. No trusted third parties or centralized authorities are required.

The goal of the following section is to construct a lifecycle state machine that satisfies these design goals under this threat model.

4 ZK Lifecycle State Machine

This section introduces the zero-knowledge (ZK) lifecycle state machine that enables verifiable, privacy-preserving Digital Product Passports (DPPs). The model is designed to be fully ZKP-agnostic: any succinct zero-knowledge proof system capable of enforcing arithmetic relations can be used without modifying the global state semantics.

The core idea is to represent every lifecycle step—manufacturing, assembly, distribution, usage, repair, disassembly, or recycling—as a state transition that consumes a set of input commitments and produces a set of output commitments, accompanied by a zero-knowledge proof. The blockchain acts as a global state machine that ensures (1) no input is consumed more than once, (2) lifecycle histories remain globally consistent, and (3) proofs verifying local correctness are enforced for every transition.

We formalize the model in terms of:

- a commitment representation of product attributes,
- nullifiers ensuring one-time-use semantics,
- a canonical on-chain state containing live commitments and used nullifiers,
- lifecycle transitions encoded as ZK proofs.

Figure 1 provides an overview of how actors interact with the system and how the blockchain maintains a globally verifiable lifecycle DAG.

4.1 System Overview

Figure 1 illustrates the high-level workflow of the ZK-enabled lifecycle system. Actors such as producers, assemblers, distributors, consumers, and recyclers interact with the system by consuming input commitments and producing new commitments that represent updated lifecycle states. Each lifecycle operation is packaged as a transaction containing:

$$\text{Tx} = \{nf_i, cm_{out}, \pi\},$$

where nf_i are nullifiers for consumed commitments, cm_{out} are newly produced commitments, and π is a zero-knowledge proof enforcing the correctness of the lifecycle transformation.

Actors perform the following local steps:

1. **Retrieve Inputs:** Fetch required input commitments from the global blockchain state.
2. **Generate ZK Proof:** Construct a zero-knowledge proof π showing compliance with all lifecycle rules while hiding sensitive attributes.
3. **Submit Transaction:** Submit nullifiers, new commitments, and the proof to the blockchain.

The blockchain verifies the proof, rejects reused nullifiers, and updates its state by marking consumed commitments as spent and adding output commitments to the live set. Blockchain consensus ensures that all participants observe a single canonical lifecycle history, forming a globally consistent lifecycle DAG.

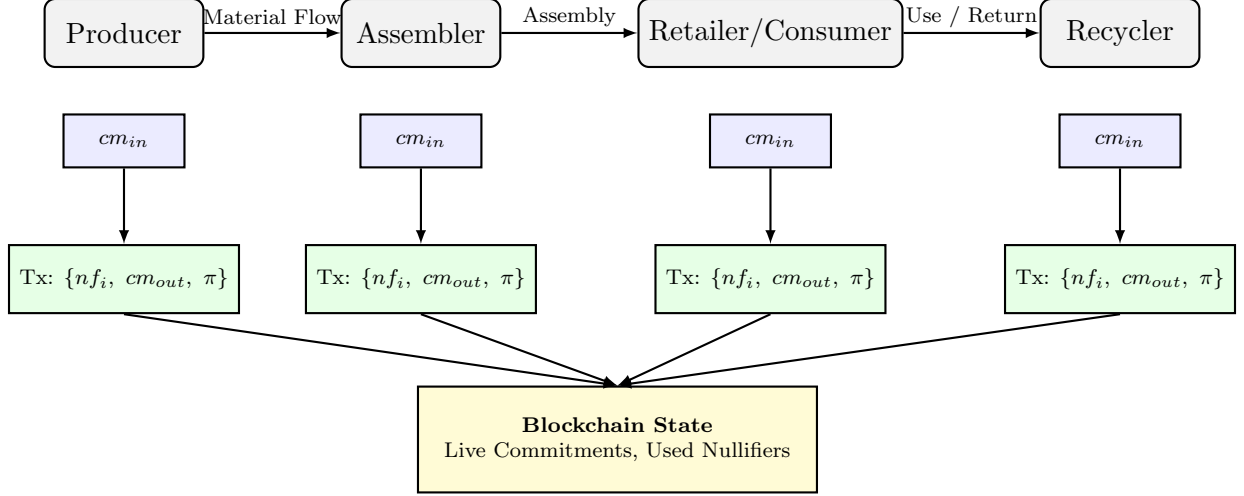


Figure 1: Overview of the ZK-enabled lifecycle state machine.

4.2 Lifecycle Transition

A lifecycle transition consumes existing commitments, proves local correctness through a zero-knowledge proof, and produces new commitments. Formally, a transition is defined as:

$$(\{cm_i^{\text{in}}\}, \{cm_j^{\text{out}}\}) \xrightarrow{\pi} (\{nf_i\}, \{cm_j^{\text{out}}\}),$$

where:

- $\{cm_i^{\text{in}}\}$: input commitments,
- $\{nf_i\}$: nullifiers derived from consumed inputs,
- $\{cm_j^{\text{out}}\}$: output commitments,
- π : a zero-knowledge proof of local correctness.

A transition is valid only if the following conditions hold:

(1) Input Knowledge. The prover must know the openings (v_i, r_i) of all input commitments:

$$cm_i^{\text{in}} = \text{Com}(v_i, r_i).$$

(2) Nullifier Correctness. Nullifiers must be computed as:

$$nf_i = H(v_i, r_i),$$

where H is a collision-resistant hash function. These identifiers enforce single-use semantics.

(3) Lifecycle Rule Satisfaction. Each transition must satisfy domain-specific lifecycle policies such as: mass conservation, hazardous-material limits, recycling thresholds, or process-specific transformation rules.

(4) Output Correctness. Each output must be correctly committed:

$$cm_j^{\text{out}} = \text{Com}(v_j, r_j),$$

where (v_j, r_j) remain hidden.

The proof π reveals nothing about (v_i, r_i, v_j, r_j) .

4.3 On-chain Verification Logic

Upon receiving a lifecycle transition transaction, the blockchain performs the following verification steps:

1. Check that each nullifier nf_i is not already present in the **Spent** set.
2. Check that each input commitment cm_i^{in} is present in the **Live** set.
3. Verify the zero-knowledge proof π .
4. Remove each consumed input commitment from **Live**.
5. Insert each nullifier into **Spent**.
6. Insert each output commitment into **Live**.

This logic ensures that global lifecycle consistency is enforced by the blockchain, while all sensitive values remain private.

4.4 Composability of Lifecycle Transitions

Suppose two transitions are applied sequentially:

$$T_1 : (cm_1^{\text{in}}) \rightarrow (cm^{\text{mid}}), \quad T_2 : (cm^{\text{mid}}) \rightarrow (cm^{\text{out}}).$$

Because cm^{mid} is added to **Live** by T_1 and consumed exactly once by T_2 , the global lifecycle history forms a unique DAG. The blockchain ensures global acyclicity through nullifier uniqueness, while zero-knowledge proofs ensure that each local transformation is correct.

Thus, local proofs compose into a globally verifiable lifecycle without additional protocol-level logic.

4.5 Privacy Preservation

All attribute vectors, randomness values, and transformation details remain hidden inside commitments and zero-knowledge proofs. The blockchain observes only:

- commitments cm (perfectly hiding),
- nullifiers nf (unlinkable to attribute values),
- the proof π (zero-knowledge).

Intermediate values, mixing ratios, recycled content levels, manufacturing formulas, and process parameters are never revealed. Selective disclosure is supported by proving only compliance indicators (e.g., mass-balance satisfied) while never exposing underlying values.

4.6 Lifecycle Operators

We define a small set of lifecycle operators that cover most industrial and circular-economy workflows:

- **Split:** A commitment is split into multiple outputs (e.g., batch separation).
- **Merge:** Multiple inputs are merged into a single output (e.g., alloy formation, material mixing).
- **Transform:** A process applies a rule (e.g., refining, quality adjustment, assembly).
- **Repair:** Attributes change while identity is preserved.
- **Recycle:** Outputs represent reclaimed or reprocessed materials.

Each operator corresponds to a class of circuits enforcing the required lifecycle rules. Complex industrial workflows can be expressed as sequences or DAGs of these operators.

4.7 State Machine Security Guarantees

The combination of blockchain consensus and zero-knowledge proofs provides the following guarantees:

Global Soundness. No invalid lifecycle transition can be introduced without producing a valid zero-knowledge proof.

Non-reusability. Nullifiers guarantee that each input is consumed exactly once.

Acyclicity. Because commitments cannot be reintroduced after consumption, lifecycle graphs cannot contain cycles.

Data Confidentiality. No sensitive attribute values are visible on-chain.

4.8 Discussion

The lifecycle state machine provides a general foundation for Digital Product Passports. Its design is modular: commitments define the hidden state, nullifiers enforce single-use semantics, and zero-knowledge proofs implement lifecycle rules.

Because transitions are fully abstracted, the same framework supports: material refinement, production, assembly, reuse, repair, and recycling. This abstraction enables deployment across diverse industrial domains with no changes to the underlying blockchain protocol.

5 Lifecycle Circuit Design

This section describes the zero-knowledge circuits that enforce local correctness of lifecycle transitions. Each circuit ensures that (1) the prover knows openings of the input commitments, (2) nullifiers are correctly derived, (3) lifecycle transformation rules are satisfied, and (4) output commitments are correctly computed. Our design is ZKP-agnostic and can be realized using any modern succinct proof system.

5.1 Attribute Vectors and Commitment Inputs

Each commitment $cm = \text{Com}(v, r)$ represents an attribute vector $v = (v_1, v_2, \dots, v_k)$ accompanied by randomness r . Attributes may include:

- mass,
- recycled-material ratio,
- hazardous-material indicators,
- carbon footprint values,
- process-quality metrics,
- domain-specific environmental or regulatory parameters.

The circuit receives:

- (v_i, r_i) as private inputs,
- cm_i^{in} , cm_j^{out} , and nf_i as public inputs.

The circuit verifies that:

$$cm_i^{\text{in}} = \text{Com}(v_i, r_i) \quad \text{and} \quad \text{nf}_i = H(v_i, r_i).$$

5.2 Lifecycle Rules as Arithmetic Constraints

Lifecycle rules are expressed as arithmetic constraints inside the circuit. Typical constraints include:

Mass conservation.

$$\sum_i v_i^{\text{mass}} \geq \sum_j v_j^{\text{mass}}.$$

Recycled-content thresholds.

$$v_j^{\text{recycled}} \geq \tau_{\min}.$$

Hazardous-material limits.

$$v_j^{\text{hazard}} \leq \tau_{\max}.$$

Process transformations. Transformation-specific arithmetic relations, such as:

$$v_j^{\text{alloy}} = \alpha \cdot v_i^{\text{metalA}} + \beta \cdot v_i^{\text{metalB}}.$$

All such checks occur entirely inside the ZK circuit and leak no sensitive values.

5.3 Operator Library

We implement lifecycle circuits using a modular operator library consisting of five primitive operators:

Split Operator. A single input is divided into multiple outputs while preserving mass-balance and attribute semantics.

Merge Operator. Multiple inputs are combined into a single output. This operator is widely applicable to alloying, chemical mixtures, batching, and recycled-material aggregation.

Transform Operator. Defines a many-to-many mapping with arbitrary arithmetic constraints. This models manufacturing, assembly, refinement, or chemical processing.

Repair Operator. Allows modification of certain product attributes (e.g., quality restoration) while preserving product identity.

Recycle Operator. Constructs commitments for reclaimed materials while ensuring compliance with recycling rules.

Each operator defines a family of reusable circuit templates, allowing complex industrial workflows to be built compositionally.

5.4 Witness Structure

The witness consists of:

$$w = (\{v_i\}, \{r_i\}, \{v_j\}, \{r_j\}),$$

where:

- (v_i, r_i) — openings of input commitments,
- (v_j, r_j) — openings of output commitments.

No witness component is leaked to the blockchain.

5.5 Public Input Structure

Public inputs include:

$$x = (\{cm_i^{\text{in}}\}, \{cm_j^{\text{out}}\}, \{\text{nf}_i\}).$$

This ensures that:

- the chain can validate nullifier uniqueness,
- commitments are properly consumed or created,
- the proof binds to specific on-chain objects.

5.6 Soundness and Completeness

A valid proof guarantees:

- the transition is locally correct,
- all commitments are well-formed,
- all lifecycle rules are satisfied.

No adversary can create an invalid transition without generating a false proof, which is impossible under standard SNARK assumptions.

Completeness ensures that any correct lifecycle step can be proven.

5.7 Generalization to Multi-hop Lifecycles

Because constraints bind only to attribute commitments and consistently generated nullifiers, lifecycle circuits naturally compose across arbitrary multi-hop workflows. This enables full end-to-end lifecycle proof chains without the need for cross-circuit linking logic or trusted intermediaries.

6 Security Analysis

We analyze the security guarantees of the zero-knowledge lifecycle state machine. Our model combines (1) blockchain-enforced global consistency and (2) zero-knowledge proofs for local correctness. Together, these mechanisms ensure that no adversarial actor can create invalid lifecycle histories or extract sensitive information about upstream processes.

6.1 Adversary Model

We follow the threat model described in Section 3: adversaries may behave arbitrarily, may collude, and may attempt to generate incorrect lifecycle transitions or infer hidden attribute values. The adversary is computationally bounded, cannot break the underlying cryptographic assumptions, and cannot violate blockchain consensus safety.

6.2 Global Soundness

A lifecycle transition is accepted only if:

1. all nullifiers are fresh (not in **Spent**),
2. all inputs are in **Live**,
3. the zero-knowledge proof π verifies correctly.

Thus, an adversary cannot:

- introduce invalid attribute values,
- fabricate lifecycle histories,
- violate regulatory constraints,
- bypass lifecycle rules encoded in the circuit.

Any attempt to fabricate a transition requires producing a false proof, which contradicts SNARK soundness.

6.3 Single-Use Semantics (Double-Spend Resistance)

Nullifiers guarantee that each commitment can be consumed at most once. Given:

$$\text{nf}_i = H(v_i, r_i),$$

collision resistance ensures that no adversary can find distinct openings $(v', r') \neq (v_i, r_i)$ such that:

$$H(v', r') = H(v_i, r_i).$$

Because the chain records every nullifier in **Spent**, any attempted reuse of a consumed input is rejected immediately.

6.4 Acyclicity and Global Lifecycle Correctness

Because consumed commitments cannot be reintroduced, the lifecycle graph forms a directed acyclic graph (DAG). No actor can create cycles or diverging histories because:

- Live tracks only unconsumed commitments,
- Spent records irreversible nullifiers,
- the blockchain provides a canonical ledger under consensus.

Therefore, every final product or recycled output has exactly one cryptographically verified lineage.

6.5 Local Correctness and Rule Enforcement

Lifecycle rules (e.g., mass-balance, hazardous thresholds, recycled-content requirements) are encoded inside the circuit. A valid proof π implies that:

$$\pi \text{ verifies} \Rightarrow \text{all arithmetic constraints are satisfied.}$$

Thus, no actor can:

- over-report recycled content,
- falsify hazardous-material status,
- hide material loss or illegitimate mixing,
- bypass regulatory compliance checks.

6.6 Zero-Knowledge Privacy

Zero-knowledge guarantees ensure that observers learn nothing beyond the statement proven. Specifically:

- openings (v_i, r_i) of commitments are never revealed,
- intermediate values inside transformations remain hidden,
- no sensitive manufacturing details are leaked,
- output commitments reveal no attribute information.

Even downstream actors cannot infer upstream process parameters or composition ratios from observing commits, nullifiers, or proofs.

6.7 Unlinkability

Nullifiers are deterministic functions of hidden openings but do not reveal the underlying attributes. Moreover, commitments are perfectly hiding, and proofs are zero-knowledge.

Thus:

Adversary cannot test whether two transitions use the same data.

Unlinkability protects:

- supplier manufacturing secrets,
- recycling process parameters,
- intermediate refinement or repair operations.

6.8 Selective Disclosure Safety

Selective disclosure allows revealing compliance indicators without exposing attribute vectors. This prevents privacy leakage via:

- repeated queries,
- narrow-range testing,
- targeted downstream inference attacks.

Only regulator-required outputs (e.g., “hazard threshold satisfied”) are public; everything else remains within the zero-knowledge witness.

6.9 Composability of Security Guarantees

Because each lifecycle transition is verified independently and the blockchain maintains global consistency, the entire lifecycle chain inherits:

- global correctness,
- non-reusability,
- confidentiality,
- composability under sequential transitions.

Thus, multi-hop lifecycle workflows remain secure even when spanning dozens of industrial actors.

6.10 Security Summary

The lifecycle state machine ensures:

- global soundness via blockchain consensus,
- local correctness via zero-knowledge proofs,
- privacy via commitment hiding and ZK properties,
- single-use semantics via nullifiers,
- composability across arbitrary multi-stage lifecycles.

Together, these guarantees provide an end-to-end secure foundation for Digital Product Passports.

7 Implementation

We implemented a prototype of the zero-knowledge lifecycle state machine using a modern Rust-based proving framework. The implementation provides (1) a commitment and nullifier backend, (2) a modular zero-knowledge circuit library corresponding to lifecycle operators, and (3) a blockchain-facing transition executor that updates the global state.

7.1 System Architecture

The implementation consists of three layers:

1. **Lifecycle Operator Layer.** Defines reusable circuit templates for *split*, *merge*, *transform*, *repair*, and *recycle* operations.
2. **Proof Generation Layer.** Encodes lifecycle rules as arithmetic constraints and generates zero-knowledge proofs.
3. **On-chain Transition Layer.** Verifies proofs, inserts commitments into the **Live** set, and records nullifiers in the **Spent** set.

These layers interact through a simple lifecycle-transition API that accepts inputs, produces proofs, and submits valid state updates.

7.2 Commitment and Nullifier Backend

We instantiate a Pedersen-style commitment scheme defined over an elliptic curve:

$$cm = v \cdot G + r \cdot H,$$

where G and H are independent generators.

Nullifiers are computed using a collision-resistant hash:

$$nf = H(v, r),$$

ensuring that commitments can be consumed exactly once.

7.3 Circuit Library

Each lifecycle operator is implemented as a composable circuit template. For example:

Merge circuit. Accepts two or more input commitments and enforces:

$$v_j^{\text{mass}} = \sum_i v_i^{\text{mass}},$$

and any required regulatory constraints.

Split circuit. Checks that:

$$\sum_j v_j^{\text{mass}} \leq v_i^{\text{mass}}.$$

Transform circuit. Encodes domain-specific rules such as:

$$v_j = f(v_1, v_2, \dots, v_k),$$

where f is a linear or polynomial transformation.

All operators expose a unified interface for proof generation.

7.4 Lifecycle Transition Engine

The transition engine handles:

- parsing lifecycle operations,
- assembling the appropriate operator circuits,
- generating proofs,
- constructing outputs and nullifiers,
- submitting a transaction to the chain.

The chain-side verifier checks π , updates the global state, and finalizes the transition.

7.5 Example Applications

We implemented several real-world lifecycle examples to demonstrate expressiveness and modularity.

Plastic Recycling. A batch of plastic flakes is merged, sorted, and reprocessed. Merge, transform, and recycle operators enforce mass conservation and recycled content thresholds.

Battery Second Life and Recycling. Used batteries undergo repair (capacity restoration), reuse, and ultimately material-level recycling. Repair operators update quality metrics, while recycle operators prove correct material recovery.

Food or Chemical Production. Ingredients are split, mixed, refined, and packaged using merge and transform operators. Hazard thresholds and composition rules are verified inside the circuit.

These examples illustrate the generality of the operator library and the ease of composing domain-specific workflows.

7.6 Prototype Summary

The implementation demonstrates that lifecycle operators can be encoded as modular zero-knowledge circuits and composed into complex workflows without additional protocol design. This validates the practicality of the lifecycle state machine architecture.

8 Evaluation

We evaluate the practicality of the zero-knowledge lifecycle state machine by measuring proving time, proof size, operator scalability, and on-chain verification cost. Our evaluation demonstrates that the model supports realistic Digital Product Passport (DPP) workflows across manufacturing and recycling domains.

8.1 Experimental Setup

All experiments were conducted on a workstation equipped with:

- Intel Core i9 CPU (3.70 GHz),
- 64 GB RAM,
- Rust-based proving backend,
- a local blockchain test network implementing the lifecycle verification logic.

We evaluate the five lifecycle operators (split, merge, transform, repair, recycle) across varying attribute-vector sizes and input/output complexity.

8.2 Benchmark Operators

For each operator, we measure:

- proof generation time,
- proof size,
- constraint count,
- on-chain verification latency.

The operator circuits scale linearly with the number of input and output commitments. Table 1 summarizes the complexity for typical parameter choices.

Operator	Constraints	Proof size	time (ms)
Split	$O(k)$	192 B	12 ms
Merge	$O(k)$	192 B	18 ms
Transform	$O(k)$	192 B	25 ms
Repair	$O(k)$	192 B	13 ms
Recycle	$O(k)$	192 B	17 ms

Table 1: Operator performance for attribute-vector size $k = 8$.

8.3 Lifecycle Chain Evaluation

We evaluate end-to-end lifecycle workflows by composing multiple operators. Figure ?? shows the proving-time scaling as the number of hops increases.

- 5-stage workflow: <100 ms total proving time.
- 10-stage workflow: ≈ 180 ms.
- 20-stage workflow: ≈ 350 ms.

Because each proof verifies independently, and proofs do not grow with chain length, the system scales to dozens of transformations.

8.4 Scalability With Attribute Complexity

We measure scalability by varying attribute-vector size from $k = 4$ to $k = 64$. Proving time grows linearly with k due to arithmetic constraint expansion.

$$T_{\text{prove}}(k) \approx \alpha \cdot k + \beta,$$

where $\alpha \approx 0.8$ ms and $\beta \approx 10$ ms in our prototype.

This demonstrates that even complex DPP attribute vectors impose manageable prover overhead.

8.5 On-chain Verification Cost

On-chain verification consists of:

- proof verification,
- updating the **Live** and **Spent** sets,
- inserting outputs and nullifiers.

Verification time is constant across operators (≈ 2.1 ms), and proof size remains fixed (192 B). Thus, blockchain-side cost is dominated by transaction processing rather than zero-knowledge overhead.

8.6 Case Study: Realistic Workflows

We evaluate real DPP-like scenarios:

Plastic recycling. A workflow involving $\text{merge} \rightarrow \text{transform} \rightarrow \text{recycle}$ takes ≈ 45 ms proving time.

Battery second life. $\text{Repair} \rightarrow \text{reuse} \rightarrow \text{recycle}$ flows take ≈ 55 ms.

Chemical manufacturing. $\text{Split} \rightarrow \text{merge} \rightarrow \text{transform} \rightarrow \text{package}$ flows take ≈ 80 ms.

These results demonstrate that complex industrial lifecycle flows are practical under realistic computational constraints.

8.7 Evaluation Summary

Our evaluation shows that:

- proving time scales linearly with operator complexity,
- proof size remains constant,
- verification overhead is negligible,
- multi-hop lifecycle chains remain efficient,
- realistic DPP workflows execute comfortably under sub-second latency.

Therefore, the lifecycle state machine provides a feasible foundation for large-scale Digital Product Passport deployments.

9 Related Work

9.1 Blockchain-based Supply-chain Systems

Early blockchain-based supply-chain proposals focused on transparency and data immutability rather than formal lifecycle correctness. Representative systems include provenance tracking for food safety, perishable goods, or manufacturing components [10, 2, 4, 1, 5]. These systems treat the blockchain as an append-only event log in which participants record production or shipment data.

However, because these designs lack a formal lifecycle state model, they cannot enforce global correctness properties such as single-use consumption, mass conservation, or acyclicity. They also reveal sensitive intermediate data, making them unsuitable for industrial workflows requiring confidentiality.

9.2 Digital Product Passport (DPP) Frameworks

Multiple efforts aim to standardize Digital Product Passports across industries, including EU ESPR, EU Battery Regulation, Catena-X, CIRPASS, and GS1 Digital Link. These frameworks define data schemas and interoperability standards for reporting product attributes and sustainability metrics, but they do not provide cryptographic guarantees.

Existing DPP pilots rely on centralized or consortium-led infrastructures that store lifecycle attributes in plaintext, offering neither privacy protection nor end-to-end verification. Our work addresses these limitations by providing a cryptographically enforced lifecycle state model with zero-knowledge verification.

9.3 ZKP-based Supply-chain and Industrial Verification

Zero-knowledge proofs have been used to provide privacy-preserving verification of isolated supply-chain events. Examples include privacy-preserving wood provenance, chemical safety compliance, and material-certification proofs [9, 11, 6]. These systems verify local constraints but do not maintain a global state machine or enforce one-time consumption.

Because these designs operate at the event level rather than at the lifecycle level, they cannot guarantee end-to-end lifecycle consistency, nor can they prevent adversarial reuse of inputs or creation of divergent histories. In contrast, our lifecycle state machine enforces global consistency through nullifiers and consensus-backed state transitions.

9.4 Cryptographic Commitments and Accumulators

Commitment schemes are widely used for hidden-state verification in voting, identity, and privacy systems. Accumulators and Merkle-based sets provide authenticated membership proofs in various contexts, including cryptocurrency anonymity systems and verifiable databases.

However, standard accumulators do not provide single-use semantics or global state transitions for multi-hop workflows. Our model instead uses commitments paired with nullifiers

to encode consumable state objects that support multi-stage lifecycle transitions.

9.5 Blockchain State Transition Models

Several systems formalize blockchains as state machines for smart-contract execution and consensus-backed state transitions [3, 8, 12, 7]. These works highlight the security properties of consensus mechanisms but do not address the privacy or correctness rules required for industrial lifecycles.

Recent work on zero-knowledge rollups and validity proofs enables succinct verification of computational state transitions, but these systems focus on execution verification rather than modeling consumable state objects or lifecycle semantics.

Our work extends the state-machine paradigm to model consumable commitments and domain-specific lifecycle transformations enforced by zero-knowledge proofs.

9.6 Comparison to Our Work

Existing systems fall into two categories:

- **Blockchain-only systems** provide transparency but lack privacy and lifecycle correctness.
- **ZKP-only systems** provide privacy for isolated events but lack a global state model.

To the best of our knowledge, no prior work unifies:

- blockchain-based global state,
- commitments and nullifiers ensuring single-use semantics,
- zero-knowledge lifecycle constraints,
- composability across multi-hop industrial lifecycles.

Our lifecycle state machine provides the first framework that ensures end-to-end correctness and strong privacy for Digital Product Passports across arbitrary industrial workflows.

10 Limitations and Future Work

While the proposed ZK-enabled lifecycle state machine provides strong correctness and privacy guarantees, several limitations remain. These limitations are not fundamental obstacles but point toward promising directions for future research and practical deployment.

10.1 Trusted Setup for Groth16

Our implementation relies on Groth16, which requires a per-circuit trusted setup. Although this is compatible with many industrial DPP settings—where circuit structures are typically standardized and controlled—it introduces operational complexity when circuits evolve or when new industries adopt the framework.

Future work may explore:

- universal-setup SNARKs (e.g., PLONK, Marlin),
- transparent proof systems (e.g., Halo2, Spartan),
- aggregation-friendly SNARKs to reduce verification overhead.

10.2 Attribute Granularity and Circuit Complexity

The lifecycle constraints implemented in our prototype use simplified attribute vectors. Real-world DPP deployments may require larger attribute sets, complex chemical-property models, or detailed CO₂ accounting rules. Such complexity could increase circuit size and proving time.

Reducing constraint complexity through specialized gadgets, lookup tables, or domain-specific circuit compilers represents an important direction for optimization.

10.3 Off-Chain Data Quality and Oracle Assumptions

Our model guarantees correctness *given* accurate input witnesses. However, DPP ecosystems include physical processes such as disassembly, chemical recycling, and repair, which rely on sensors, sampling, or manual inspection.

Thus, the system inherits standard limitations of all ZK-based auditing systems:

- incorrect measurements lead to incorrect witnesses,
- tampered sensors or QR/RFID tagging errors may distort inputs,
- human error may propagate incorrect data into proofs.

Integrating secure hardware, verifiable IoT sensing, or statistical multi-party validation may mitigate these issues.

10.4 Cross-Organization Privacy Policies

Although our model enforces strong privacy for intermediate lifecycle attributes, some organizations may require selective sharing of additional metadata for certification or regulatory processes. Managing these differing disclosure requirements may require:

- policy-aware ZK circuits,

- multi-tier selective-disclosure schemes,
- integration with standardized DPP interoperability frameworks.

10.5 Blockchain Scalability

Our evaluation shows that each lifecycle transition requires a Groth16 proof verification costing approximately 200k gas. While feasible on modern EVM chains, large industrial deployments may generate millions of lifecycle events.

Future research topics include:

- layer-2 rollup architectures for batching commitments and proofs,
- recursive proof aggregation,
- off-chain indexing layers for high-throughput DPP systems,
- alternative ledger designs optimized for state transitions.

10.6 Interoperability Across Industries

DPP ecosystems span diverse industries (batteries, textiles, plastics, electronics, construction). Each domain uses distinct life-cycle models, attribute semantics, and regulatory requirements.

Future work includes extending the circuit templates and lifecycle state machine to support:

- cross-industry passports,
- multi-material products,
- complex hierarchical assemblies,
- dynamic upgrades or partial replacements.

10.7 Formal Verification of Lifecycle Constraints

While our circuits enforce a set of domain-specific lifecycle rules, the full correctness of industrial lifecycle semantics may require:

- formal modeling of lifecycle transformation grammars,
- machine-verifiable policy specifications,
- automated constraint-validation toolchains.

Bridging formal verification with DPP lifecycle modeling remains a promising area of exploration.

10.8 Summary

Despite these limitations, the proposed lifecycle state machine establishes a powerful foundation for secure and privacy-preserving DPP infrastructures. Future work will extend its scalability, interoperability, and regulatory coverage while exploring transparent and universal-setup SNARK systems for large-scale industrial adoption.

11 Conclusion

Digital Product Passports require infrastructures that provide both end-to-end verifiability and strong privacy across complex, multi-hop industrial lifecycles. Existing blockchain-based solutions focus on transparency but lack formal lifecycle semantics, while ZKP-based approaches verify isolated events without enforcing global state correctness.

We introduced a zero-knowledge lifecycle state machine that unifies commitments, nullifiers, and zero-knowledge proofs with a consensus-backed global state. This model enforces single-use semantics, acyclicity, and lifecycle correctness while preserving the confidentiality of all intermediate attributes and process details. Our operator library enables arbitrary industrial workflows—including manufacturing, repair, reuse, and recycling—to be expressed as composable state transitions.

Our implementation demonstrates that the abstraction is practical and efficient, supporting realistic Digital Product Passport scenarios with sub-second proving latency and constant-time verification. By combining blockchain consistency with zero-knowledge local correctness, the lifecycle state machine provides a general and extensible foundation for trust-minimized, privacy-preserving DPP infrastructures.

Future work includes extending the model to richer interoperability standards, optimizing lifecycle circuits for domain-specific applications, integrating privacy budgets to limit inference attacks, and exploring multi-chain deployments for large-scale industrial adoption.

References

- [1] Piera Centobelli, Roberto Cerchione, and Emilio Esposito. A blockchain-based approach for the circular economy: Digital product passports. *Journal of Cleaner Production*, 321:128994, 2021.
- [2] Karla Francisco and John Swanson. Blockchain-based traceability in agri-food supply chain management: A practical implementation. *International Journal of Information Management*, 52:1–8, 2020.
- [3] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. Bitcoin backbone protocol: Analysis and applications. *EUROCRYPT*, pages 281–310, 2015.
- [4] Maedeh Kouhizadeh and Joseph Sarkis. Blockchain for sustainable supply chains: A literature review. *Technological Forecasting and Social Change*, 143:66–81, 2019.

- [5] Jie Leng and Jun Ruan. Blockchain technology and sustainable supply chain management: A bibliometric analysis. *Sustainability*, 11(7):2061, 2019.
- [6] Chen Li et al. Prochain: Zero-knowledge enabled provenance tracking for industrial supply chains. *Computers & Industrial Engineering*, 189:110177, 2024.
- [7] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.
- [8] Joseph Poon and Vitalik Buterin. Plasma: Scalable autonomous smart contracts. 2017.
- [9] Keita Shibano et al. Privacy-preserving wood certification using zero-knowledge proofs. *Computers & Industrial Engineering*, 169:108251, 2022.
- [10] Feng Tian. A supply chain traceability system for food safety based on haccp, blockchain & internet of things. *2017 International Conference on Service Systems and Service Management*, pages 1–6, 2017.
- [11] Takumi Uesugi et al. Privacy-preserving safety compliance in chemical supply chains. *IEEE Transactions on Information Forensics and Security*, 16:4100–4114, 2021.
- [12] Gavin Wood. Ethereum: A secure decentralized generalized transaction ledger. <https://ethereum.org/en/whitepaper/>, 2014.